

基于 OpenAI Agents SDK 的面试型智能体工作台需求文档

Executive Summary

本项目建议定位为一个“可对话、可切换模型、可观察 Agent 执行过程、可落库回放”的智能体工作台：前端使用 React 提供类 ChatGPT 界面，后端使用 FastAPI 封装 OpenAI Agents SDK，SQLite 作为会话、运行、事件与图片资产的统一存储层。OpenAI Agents SDK 的核心原语本身就覆盖了本项目最需要展示的几项能力：

Agent、Runner、tools、handoffs、streaming、results、sessions 与 tracing。¹

从面试展示角度，最有价值的不是“单纯做一个聊天框”，而是展示你理解了 Agent 的真实运行机制：总控 Triage 如何分流，何时用“manager/agents-as-tools”，何时用 handoff，工具调用如何审计，流式事件如何可视化，为什么多模型统一走 OpenAI-compatible 的 Chat Completions 形态，以及为什么没有 OpenAI 官方 Key 时要关闭默认 tracing 或改用独立 tracing key。SDK 官方文档也明确说明：许多第三方模型提供方仍未完整支持 Responses API，因此示例里常用 OpenAIChatCompletionsModel 来对接兼容端点；同时 tracing 默认开启，且默认沿用 OpenAI key。²

在模型接入上，GLM、MiniMax、Qwen 已经都提供了 OpenAI-compatible 接口；NVIDIA NIM 提供 OpenAI-compatible 的 LLM 与 Image Generation 接口，并明确把 FLUX.1/FLUX.2 系列作为支持模型；Black Forest Labs 官方 FLUX API 则采用“提交任务、轮询结果”的异步模式，因此更适合作为独立图片工具适配器，而不是直接拿来当文本推理模型。³

建议的最小可行版本是：一个总控 TriageAgent，加上 TechAgent、EcommerceAgent、ImageAgent、ModelJudgeAgent 四类特色子能力；流式事件通过 SSE 映射为前端时间线；所有会话、运行、工具调用、原始响应与图片资产落 SQLite；图片生成优先走 NVIDIA NIM 的 OpenAI-compatible image endpoint，Black Forest Labs 作为 v1 扩展。这样既能覆盖 SDK 官方推荐的 routing / agents-as-tools / parallel execution / LLM-as-a-judge / HITL 等模式，又能形成非常完整的面试演示链路。⁴

项目概述

一句话定位：一个面向开发者和面试场景的多模型智能体工作台，支持总控分流、工具调用、流式事件可视化、模型对比、图片生成、会话回放与运行审计。

目标受众主要有两类。第一类是面试官和技术评审，他们关心你是否真正理解 Agent 的编排、工具、事件流、审计与安全；第二类是你自己作为开发者，需要一个可以持续扩展的实验台，把不同 OpenAI-compatible 模型统一接入、做对比和沉淀经验。OpenAI Agents SDK 官方把 Agent 定义为“带 instructions、tools、handoffs、guardrails、structured outputs 的 LLM 单元”，而 Runner 则负责驱动“模型输出—工具调用—重新入环—handoff—终止”的运行循环；这正好适合作为一个“可视化工作台”的后端内核。⁵

面试亮点建议集中在五件事。第一，你不是只会调模型，而是会搭 Agent runtime：知道 Runner.run_streamed()、stream_events()、new_items、raw_responses、last_agent、to_state() 分别适合什么场景。第二，你清楚 manager 与 handoff 两种多 Agent 模式的边界。第三，你能把第三方模型接到 SDK 上，而不是只能用 OpenAI 官方模型。第四，你做了可观测性和审计：会话、事件、工具、token、trace 都能回看。第五，你考虑了安全：prompt injection、防滥用、高危写操作审批与最小权限。⁶

本项目的架构前提里，有几项信息当前**未指定**。为了让文档可执行，建议先按下表作为默认假设推进：

项目	当前状态	建议默认值
认证方式	未指定	MVP 走单用户本地模式；v1 再加 JWT / Cookie Session
并发用户数	未指定	按 1-10 个演示用户设计
部署环境	未指定	本地开发 + Docker Compose 单机部署
文件存储	未指定	先本地磁盘 + SQLite；v1 再抽象对象存储
第三方模型预算	未指定	默认优先接入免费/试用额度模型
是否需要多租户	未指定	MVP 不做多租户

从能力边界看，本项目**不建议**在 MVP 阶段做复杂账号体系、团队协作、企业 RBAC、真正的生产级 RAG 知识库编排或复杂文件上传流水线。更合理的策略是：先把“会话—运行—事件—工具—模型—图片资产”这条最核心的 Agent 演示主线打通，再逐步扩展。SDK 官方自己的 examples 也说明，routing、agents-as-tools、parallel execution、LLM as judge、guardrails、streaming guardrails、HITL 都是可以循序渐进加入的模式。⁷

功能分级与核心用例

功能清单

建议按 MVP / v1 / v2 三层推进。这样既符合面试展示节奏，也与 SDK 的学习曲线一致。

优先级	功能	说明
MVP	Chat 工作台基础 UI	左侧会话列表，主区域消息流，输入框，停止生成，重新生成
MVP	Triage 总控 Agent	统一入口，负责路由到 Tech/Ecommerce/Image/ModelJudge
MVP	单模型聊天	支持 GLM、MiniMax、Qwen 中至少 2-3 个文本模型
MVP	流式事件时间线	展示 token delta、agent 切换、tool 调用、tool 输出、run 完成
MVP	会话与运行落库	conversations / messages / agent_runs / run_events / tool_calls
MVP	模型切换	每次对话可指定 primary model
MVP	图片生成	支持一个 FLUX 通道并保存资产记录
MVP	调试抽屉	查看 raw events、usage、最终 agent、tool 参数与输出
v1	多模型并发对比	问题 fan-out 到多个模型，并用 ModelJudge 总结
v1	图片画廊	查看历史图片、重新生成、引用历史提示词
v1	手动模式切换	Auto / Tech / Ecommerce / Image / Compare 模式
v1	输入/输出 guardrails	注入检测、越权工具过滤、输出结构校验
v1	SDK session 可选接入	conversation_id= session_id，支持多轮上下文自动注入
v1	LLM as judge 评分卡	从“质量 / 速度 / 成本 / 风格匹配”几个维度评分

优先级	功能	说明
v2	写操作审批	HITL 审批 UI、暂停 / 恢复 Run
v2	对话分支	参考 AdvancedSQLiteSession 的 conversation branching 思路
v2	文件上传与分析	让 TechAgent / EcommerceAgent 支持针对文件工作
v2	WebSocket 双向控制	支持前端中途确认、取消、修改 run 参数
v2	统一评估集	固定 case 回归测试、成功率与成本趋势图

之所以这样划分，是因为 OpenAI Agents SDK 官方示例已经覆盖了其中的大部分模式：routing、parallel execution、agents as tools、LLM as judge、input/output guardrails、streaming guardrails、human-in-the-loop with state serialization。把这些模式映射成产品功能，天然就具备“边做边学”的价值。 ⁷

Agent 设计

建议采用“**总控 + 专家 + 比较器**”的混合式多 Agent 架构。官方文档对多 Agent 给出了两种通用模式：一是 manager (agents as tools)，由中央控制者保留对话主导权；二是 handoffs，把控制权转交给专业 Agent。对于你的工作台，MVP 推荐以 **manager 模式为主**，因为它更适合保持统一用户体验、单一聊天窗口与稳定时间线；当用户进入持续的专业会话时，再引入 handoff。 ⁸

Agent	角色	触发条件	推荐协作方式	结构化输出
TriageAgent	总控路由、选择 specialist、决定是否 compare / image	默认接收所有用户输入	manager 为主，必要时 handoff	RouteDecision
TechAgent	解释报错、生成代码思路、技术问答	用户显式选 Tech 或输入带代码/异常/技术请求	先作为 tool；持续会话再 handoff	TechResponse 可选
EcommerceAgent	电商标题检测、违规词提示、卖点重写、短文案润色	用户显式选 Ecommerce 或识别为商品文案任务	先作为 tool	EcomReview
ImageAgent	生成图片、优化提示词、列出历史图片	用户显式选 Image 或包含“画图/生成图片/海报”意图	作为 tool	ImagePlan
ModelJudgeAgent	多模型对比、评分、总结	compare_models 非空或用户请求“比较模型”	专家 tool + 并发 orchestration	JudgeReport
GeneralAgent	常规闲聊、兜底回复	无明显专家意图	直接作答	无

这里最建议重点展示的是 **TriageAgent 的结构化路由**。SDK 官方说明：当给 Agent 配置 output_type 时，模型会走 structured outputs，而不是自由文本。你可以把路由决策设计成一个 Pydantic 模型，例如 intent、specialist、orchestration_mode、compare_models、needs_image、confidence，让 Triage 先产出结构化 JSON，再由后端根据 JSON 决定下一步执行策略。这既是“Agent 核心知识理解”的体现，也符合 OpenAI 官方关于“用 structured outputs 限制风险传播”的建议。 ⁹

详细用例与触发逻辑

TriageAgent

TriageAgent 是所有交互的统一入口。建议决策顺序如下：先看前端是否显式选择了模式；如果显式选择，则走确定性路由，不再让模型猜；否则由 TriageAgent 以结构化输出模式给出 `RouteDecision`。如果结果是 `compare`，后端进入模型对比流水线；如果结果是 `image`，调用 ImageAgent；如果结果是 `tech` 或 `ecommerce`，优先把相应 Agent 当作 tool 调用，让总控保留对话控制权；只有当用户明确进入一个长期专业主题时，才使用 handoff 切换当前 agent。官方文档对这两种模式的区别给出的定义非常清晰：manager 是中央编排，handoff 是把会话控制权正式转交。 ¹⁰

TechAgent

TechAgent 的核心价值不在于“写很多代码”，而在于把技术问题回答做成一个可观察的工具 workflow。建议 MVP 里放三个工具：`explain_error_signature`、`search_tech_notes`、`summarize_patch_plan`。其中第一个工具可对 traceback / error message 做模式匹配；第二个工具可查本地技术笔记或内置知识片段；第三个工具可把模型草稿整理成结构化修复步骤。这样可以明显展示：模型不是直接乱答，而是在调用工具、拿到结果、再生成最终回复。SDK 官方的 run loop 也是这么工作的：LLM 先输出；若有 tool calls 就执行工具并重新入环；若有 handoff 就切换 agent 并重跑。 ¹¹

EcommerceAgent

EcommerceAgent 是这个工作台最适合面试演示的垂直特色。建议工具链采用“规则优先，模型润色”的方式：先用 `check_sensitive_words` 执行本地规则校验，再用 `extract_selling_points` 做结构化卖点抽取，最后由模型根据规则结果和卖点结果生成“问题清单 + 优化版本 + 修改理由”。Qwen 的 function calling 文档给出了非常实用的最佳实践：不要把完整工具库都交给模型，候选工具集尽量控制在 20 个以内；危险工具不要直接暴露；高权限或不可逆操作必须人工确认。即便你的电商文案工具并不算高危，这种设计思想本身很适合拿来说明你对 Agent 工程治理的理解。 ¹²

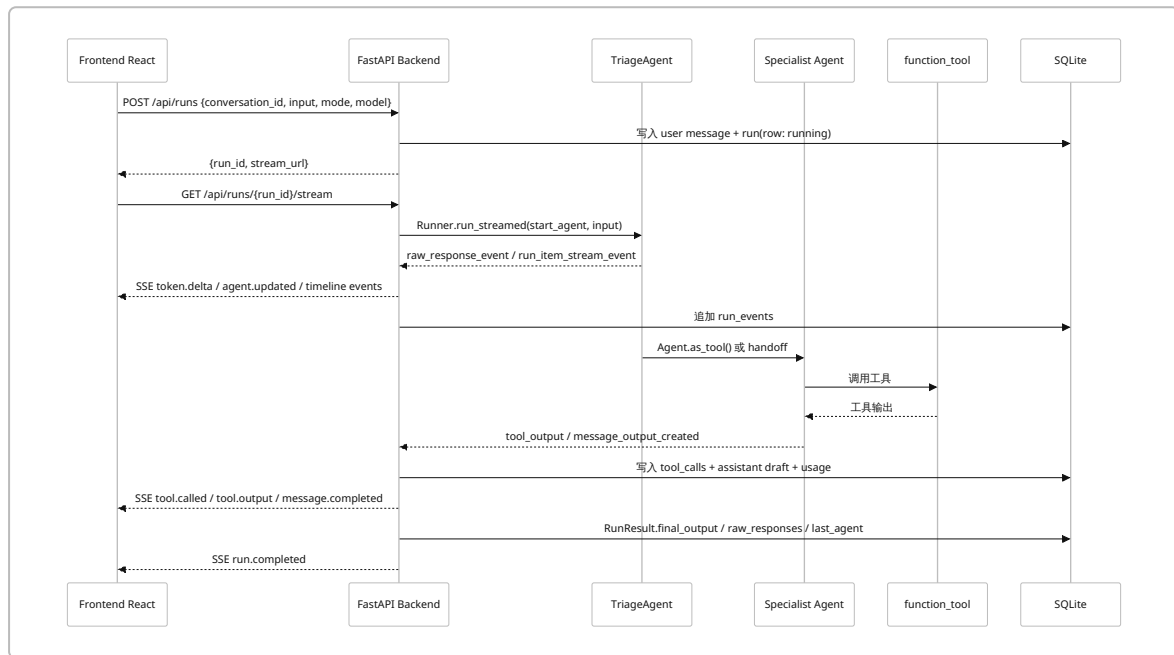
ImageAgent

ImageAgent 建议拆成三步。先由一个轻量级 Agent 或结构化提取器把用户自由描述转成 `ImagePlan`，字段包括 `subject`、`style`、`aspect_ratio`、`negative_prompt`、`brand_constraints`。再由 `optimize_image_prompt` 生成更适合目标图片模型的最终提示词。最后调用 `generate_image` 工具，由图片提供方生成资产并返回 `asset_id`、存储位置、缩略图信息。NVIDIA NIM 明确提供 OpenAI-compatible 的 Image Generation API，并列出了 `flux.1-dev`、`flux.1-schnell`、`flux.2-klein-4b` 等模型；Black Forest Labs 官方文档则说明其 API 是异步的，需要先创建生成任务，再轮询结果。因此在你的设计里，ImageAgent 只需关心“调用图片工具”，具体后端适配器可以隐藏实现差异。 ¹³

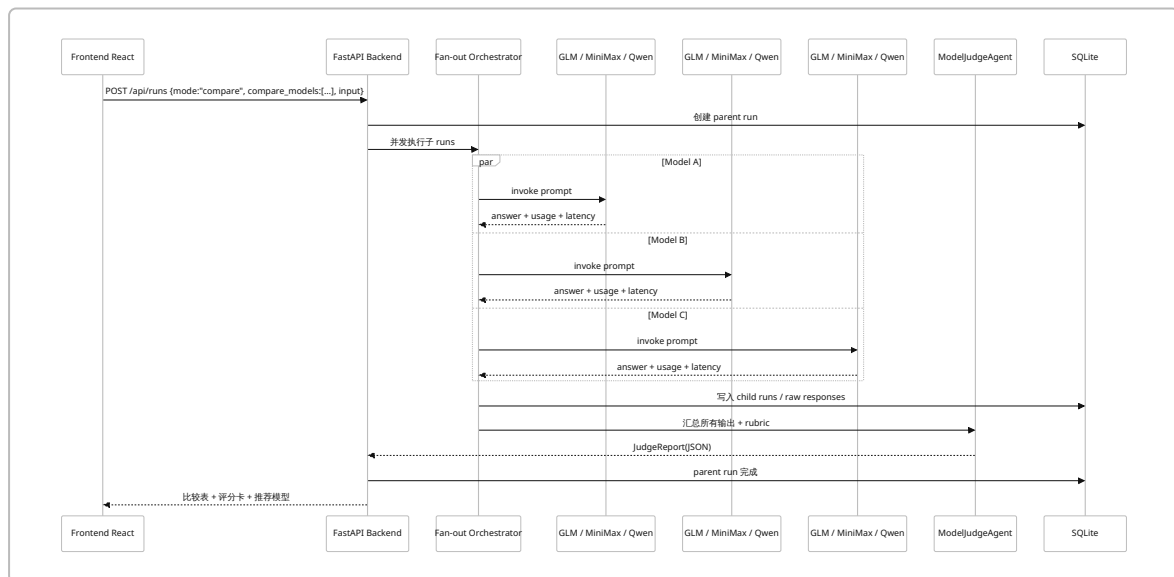
ModelJudgeAgent

ModelJudgeAgent 是“多模型工作台”区别于普通聊天工具的关键。建议它不要直接手写逻辑拼接，而是走一个标准化流程：接收同一个 prompt 和 compare model 列表；后端并发 fan-out 到各模型；记录每个模型的答案、耗时、usage 与错误状态；再把这些结果喂给 ModelJudgeAgent，让它按固定 rubric 输出评分与总结。OpenAI Agents SDK 官方 examples 明确包含 parallel agent execution 和 LLM as judge；从产品表达上，把两者结合成“Compare + Judge”面板，和你的模型池能力就自然契合了。 ⁷

关键时序图



这个时序图对应的是 SDK 官方 streaming 与 results 机制：run_streamed() 会产生 raw_response_event、run_item_stream_event 与 agent_updated_stream_event；而在流结束后，RunResultStreaming 会持有完整的 final_output、new_items、raw_responses、last_agent 等结果面。 14



这个 compare 流程是对 SDK “parallel execution + judge” 模式的产品化包装：前半段是并发跑多个子任务，后半段用结构化 JudgeReport 做归纳。其面试亮点在于：不仅比较“谁答得好”，还可以展示 latency、token usage、失败降级与最终推荐依据。SDK 的 usage 文档也明确说明，run 级别和 per-request usage 都可被访问与记录。 15

系统方案

后端设计

从 SDK 机制看，后端有三个关键职责。第一，**Agent orchestration**：根据 mode 和 route decision 选择 start agent、tool 集和 model provider。第二，**stream normalization**：把 SDK 原始事件转换为统一 SSE 事件。第三，**run persistence**：把 `final_output`、`new_items`、`raw_responses`、`usage`、`last_agent` 与 tool 审计信息写入 SQLite。官方 results 文档明确指出：`new_items` 最适合做日志、UI、审计与调试，`raw_responses` 最适合做 provider-level 诊断，`last_agent` 适合做下一轮默认 agent。¹⁶

MVP 不建议强依赖 provider 的 server-managed conversation 机制。更稳妥的做法是：**应用自己管理 conversations/messages/agent_runs**，必要时再把 `conversation_id` 同步为 SDK `session_id`。这是因为官方 sessions 文档明确说明：`session` 不能和 `conversation_id`、`previous_response_id`、`auto_previous_response_id` 混用；而你的工作台又需要跨 GLM/MiniMax/Qwen/NIM 统一行为，应用级本地状态会更可控。¹⁷

API 列表

建议使用“**POST 创建 Run + GET SSE 订阅流**”的两段式接口，而不是一个直接 POST 返回 SSE。原因很简单：`EventSource` 打开的是一个长期 HTTP 连接，用于接收 `text/event-stream`；而浏览器侧的 SSE 模式本质上适合 GET 订阅。也就是说，如果你决定采用 SSE，最干净的 API 形态就是“先创建 run，再订阅 stream”。当未来需要真正的双向交互（例如审批、动态补充参数、协作式控制）时，再升级到 WebSocket。¹⁸

路径	方法	用途	请求示例	响应示例
<code>/api/health</code>	GET	健康检查	无	<code>{"ok": true}</code>
<code>/api/models</code>	GET	获取可用模型配置列表	无	<code>[{id, display_name, provider}]</code>
<code>/api/conversations</code>	GET	获取会话列表	<code>?limit=20&cursor=...</code>	<code>{"items": [...], "next_cursor": ...}</code>

路径	方法	用途	请求示例	响应示例
<code>/api/conversations</code>	POST	创建新会话	<code>{"title": "新会话", "mode": "auto"}</code>	<code>{"id": "conv_xxx"}</code>
<code>/api/conversations/{id}</code>	GET	获取会话详情	无	<code>{"id": "conv_xxx", "title": "..."} </code>
<code>/api/conversations/{id}/messages</code>	GET	获取消息历史	无	<code>{"items": [...messages...]} </code>
<code>/api/runs</code>	POST	创建一次新运行	见下方主示例	<code>{"run_id": "run_xxx", "stream": "stream"}</code>
<code>/api/runs/{id}</code>	GET	获取运行结果	无	<code>{"status": "completed", "final": {...}} </code>
<code>/api/runs/{id}/stream</code>	GET	SSE 流式事件	无，浏览器订阅	<code>event: token.delta</code> 等
<code>/api/runs/{id}/events</code>	GET	回放事件时间线	无	<code>{"items": [...run_events...]} </code>

路径	方法	用途	请求示例	响应示例
<code>/api/runs/{id}/cancel</code>	POST	取消运行	无	<code>{"status": "cancelling"}</code>
<code>/api/runs/{id}/approve</code>	POST	审批高危工具调用	<code>{"tool_call_id": "call_xxx", "decision": "approve"}</code>	<code>{"status": "resumed"}</code>
<code>/api/assets</code>	GET	图片资产列表	<code>?conversation_id=...</code>	<code>{"items": [...assets...]}</code>
<code>/api/assets/{id}</code>	GET	图片资产详情	无	<code>{"id": "asset_xxx", "prompt": "..."}</code>

主入口建议如下：

```
POST /api/runs
{
  "conversation_id": "conv_01JY...",
  "input": "请比较 GLM、MiniMax、Qwen 写一段电商标题的效果",
  "mode": "auto",
  "primary_model_id": "glm-5.2",
  "compare_model_ids": ["glm-5.2", "MiniMax-M3", "qwen-plus"],
  "debug": true,
  "attachments": []
}
```

```
200 OK
{
  "run_id": "run_01JY...",
  "stream_url": "/api/runs/run_01JY.../stream"
}
```


其背后的核心实现，与 SDK 的官方模型接口和运行机制是一致的：`Runner.run()` / `Runner.run_streamed()` 负责驱动 loop；流结束后读取 `RunResultStreaming` 的完整结果面。¹⁹

流式接口与事件格式

推荐 **SSE 作为 MVP 的默认流式协议**。理由不是“WebSocket 不好”，而是你的首个演示系统主要需要的是“后端不断向前端推送 token 与事件”，而不是高度双向交互。MDN 对 EventSource 的定义是：前端通过 `EventSource` 打开一个持久 HTTP 连接，服务器以 `text/event-stream` 格式持续推送事件；web.dev 也明确指出 SSE 是单向通道，而 WebSocket 更适合全双工场景。²⁰

MVP 建议把 SDK 的三类流事件映射为前端可消费的统一事件名：

SDK 事件	规范化前端事件	用途
<code>raw_response_event</code>	<code>token.delta</code> / <code>response.raw</code>	实时渲染 token，必要时保留原始 provider 事件
<code>run_item_stream_event</code> + <code>tool_called</code>	<code>tool.called</code>	展示“调用了哪个工具、参数是什么”
<code>run_item_stream_event</code> + <code>tool_output</code>	<code>tool.output</code>	展示工具输出摘要
<code>run_item_stream_event</code> + <code>message_output_created</code>	<code>message.completed</code>	一条 assistant message 成型
<code>run_item_stream_event</code> + <code>handoff_requested</code> / <code>handoff_occured</code>	<code>handoff.requested</code> / <code>handoff.done</code>	展示 agent 转移
<code>run_item_stream_event</code> + <code>reasoning_item_created</code>	<code>reasoning.created</code>	调试模式展示 reasoning item
<code>agent_updated_stream_event</code>	<code>agent.updated</code>	UI 上切换当前 agent badge
流结束 + <code>RunResultStreaming</code>	<code>run.completed</code>	一次运行正式结束
异常	<code>run.error</code>	错误 toast + 时间线红色节点

一个推荐的 SSE 数据帧如下：

<pre>event: tool.called data: {"runId":"run_01JY","seq":18,"agent":"EcommerceAgent","tool":"check_sensitive_words","arguments":{"platform":"taobao","text":"..."}}</pre>
<pre>event: token.delta data: {"runId":"run_01JY","seq":19,"agent":"EcommerceAgent","delta":"建议把“史上最低”改为“超值优惠" }</pre>

SDK 官方 streaming 文档说明得非常清楚：`run_streamed()` 返回 `RunResultStreaming`，`stream_events()` 是一个 async stream；你可以在事件流里看 `raw_response_event`、`agent_updated_stream_event` 和 `run_item_stream_event`。同时，results 文档也强调：**必须把 `stream_events()` 消费到结束，run 才算真正确认完成**，因为 `final_output`、`raw_responses`、session 持久化等收尾工作可能在最后一个可见 token 之后才完成。 ²¹

Agent 执行结果与应用侧 RunResult 结构

SDK 的结果面可以直接映射到你的应用层数据结构：

SDK 字段	应用层含义	落库位置	前端用途
<code>final_output</code>	最终回复文本 / JSON	<code>messages</code> 、 <code>agent_runs.final_output_*</code>	聊天主消息
<code>new_items</code>	工具、handoff、approval、message 元数据	<code>run_events</code> 、 <code>tool_calls</code>	时间线、执行面板
<code>raw_responses</code>	provider 原始模型返回	<code>agent_runs.raw_responses_json</code>	调试查看、排障
<code>last_agent</code>	本轮最终处理 agent	<code>conversations.last_agent_name</code>	下一轮默认 specialist
<code>interruptions</code> / <code>to_state()</code>	审批中断与可恢复状态	<code>agent_runs.serialized_state</code>	v2 审批恢复
<code>context_wrapper.usage</code>	run 级 token / request 统计	<code>agent_runs.usage_json</code>	成本、延迟、对比视图

这些字段并不是你“自己发明出来的概念”，而是 SDK 官方结果文档明确推荐的消费面：`final_output` 用于最终答案，`new_items` 用于 UI / logs / audits / debugging，`raw_responses` 用于原始模型诊断，`last_agent` 用于下一轮默认 agent，`interruptions` 和 `to_state()` 用于 pause/resume。 ¹⁶

数据库设计

SQLite 作为 MVP 存储是合适的，因为它足够轻量、便于本地演示，同时 SDK 自己也提供了 `SQLiteSession` 与 `AdvancedSQLiteSession`；前者支持持久会话历史，后者支持分支、使用分析与结构化查询。对你的工作台来说，更推荐应用自建表作为“主存储”，把 SDK session 作为可选增强，而不是反过来。 ²²

表结构

表名	关键字段	说明
<code>conversations</code>	<code>id</code> , <code>title</code> , <code>mode</code> , <code>session_id</code> , <code>last_agent_name</code> , <code>created_at</code> , <code>updated_at</code> , <code>archived</code> , <code>metadata_json</code>	会话主表
<code>messages</code>	<code>id</code> , <code>conversation_id</code> , <code>role</code> , <code>content_text</code> , <code>content_json</code> , <code>agent_name</code> , <code>model_name</code> , <code>sequence_no</code> , <code>parent_run_id</code> , <code>created_at</code>	用户/助手/工具消息历史

表名	关键字段	说明
agent_runs	id, parent_run_id, conversation_id, entry_agent, final_agent, provider, model_id, trace_id, thread_id, status, started_at, ended_at, final_output_text, final_output_json, usage_json, raw_responses_json, serialized_state, error_text, run_config_json	一次 Agent 运行主记录
run_events	id, run_id, seq, event_type, sdk_event_type, event_name, agent_name, tool_name, payload_json, delta_text, created_at	流式/语义事件流水
tool_calls	id, run_id, tool_call_id, tool_name, tool_kind, arguments_json, status, output_preview, output_json, latency_ms, approved_by, created_at, finished_at	工具调用审计
model_configs	id, display_name, provider, modality, api_shape, base_url, model_id, api_key_env, capabilities_json, extra_body_defaults_json, timeout_ms, priority, enabled, created_at, updated_at	模型池配置
generated_assets	id, conversation_id, run_id, model_config_id, provider, prompt, revised_prompt, mime_type, storage_uri, thumbnail_uri, metadata_json, created_at	图片资产与画廊

字段定义建议

表	字段	类型	说明
conversations	id	TEXT PK	业务主键，建议 UUIDv7
conversations	session_id	TEXT NULL	若启用 SDK session，则等于 conversation id
messages	role	TEXT	user / assistant / system / tool
messages	content_json	TEXT NULL	结构化 message / 引用 / multimodal 内容
agent_runs	trace_id	TEXT	本次 run 的关联 ID，贯穿日志与 SSE
agent_runs	thread_id	TEXT	建议直接使用 conversation_id
agent_runs	serialized_state	TEXT NULL	HITL 中断恢复所需快照
run_events	event_type	TEXT	规范化事件名，如 token.delta
run_events	sdk_event_type	TEXT	原始 SDK 事件，如 raw_response_event
tool_calls	tool_call_id	TEXT	来自 ToolContext.tool_call_id
tool_calls	status	TEXT	pending/running/success/error/rejected
model_configs	api_shape	TEXT	chat_completions / responses / nim_image / bfl_async

表	字段	类型	说明
model_configs	capabilities_json	TEXT	{"tools":true,"stream":true,"image_gen":false}
generated_assets	storage_uri	TEXT	本地路径或对象存储 URI

SDK 的上下文文档明确说明，`ToolContext` 会暴露 `tool_name`、`tool_call_id` 与 `tool_arguments`，这正适合作为 `tool_calls` 的主记录来源。²³

示例记录

```
{
  "conversations": {
    "id": "conv_01JY0T6M4J9W7F",
    "title": "比较 GLM / Qwen / MiniMax 写电商文案",
    "mode": "auto",
    "session_id": "conv_01JY0T6M4J9W7F",
    "last_agent_name": "ModelJudgeAgent"
  },
  "agent_runs": {
    "id": "run_01JY0TB8WQJ5X2",
    "parent_run_id": null,
    "conversation_id": "conv_01JY0T6M4J9W7F",
    "entry_agent": "TriageAgent",
    "final_agent": "ModelJudgeAgent",
    "provider": "multi",
    "model_id": "glm-5.2",
    "trace_id": "trace_01JY0TB8Y9A4V8",
    "thread_id": "conv_01JY0T6M4J9W7F",
    "status": "completed"
  },
  "tool_calls": {
    "id": "tc_01JY0TC2B6QG9P",
    "run_id": "run_01JY0TB8WQJ5X2",
    "tool_call_id": "call_6596dafa2a6a46f7a217da",
    "tool_name": "check_sensitive_words",
    "status": "success",
    "latency_ms": 37
  },
  "generated_assets": {
    "id": "asset_01JY0V5Q9Y8M3K",
    "conversation_id": "conv_01JY0T6M4J9W7F",
    "run_id": "run_01JY0V4P6J2DD1",
    "provider": "nvidia_nim",
    "model_config_id": "model_flux_schnell",
    "prompt": "极简深色科技风工作台封面",
    "storage_uri": "file:///data/assets/asset_01JY0V5Q9Y8M3K.png"
  }
}
```

前端设计

前端建议做成一个单页工作台，而不是多页 CRUD 管理后台。因为你要展示的是“一个运行中的 Agent 系统”，不是普通业务页面。

页面/组件	作用	MVP 是否必须
AppShell	整体布局	是
ConversationSidebar	左侧会话列表 + 新建会话	是
ChatWindow	主消息流区域	是
Composer	输入框、停止生成、重试	是
ModelSelector	选择 primary model / compare list / mode	是
AgentBadge	显示当前 active agent	是
ExecutionDrawer	右侧执行面板总容器	是
EventTimeline	展示 token / tool / handoff / usage 节点	是
ToolCallCard	显示工具参数与结果摘要	是
RawResponseViewer	显示原始 raw response JSON	是
ComparePanel	并排展示多模型输出 + Judge 评分	v1
ImageGallery	展示历史图片资产	v1
ApprovalDialog	显示待审批工具调用	v2

消息区建议只显示“用户实际看到的内容”，而工具与事件全部折叠进右侧 `ExecutionDrawer`。这样既符合 ChatGPT 类交互习惯，又不会让调试信息污染主内容。真正的面试亮点在于：**任意一条 assistant message，都可以展开看到它背后的 agent 切换、tool 调用、raw events 和 usage。**

前端状态流

推荐的前端状态流如下：用户发送消息后，先乐观写入 user message；随后调用 `POST /api/runs` 拿到 `run_id`；开启 `EventSource` 订阅；`token.delta` 更新临时 assistant buffer；`tool.called` 与 `tool.output` 写进 timeline；`agent.updated` 切换 badge；`run.completed` 时把临时 assistant buffer 固化成正式一条 message，同时刷新 run 概览与 usage 面板。整个过程中，`run_events` 的主键可直接作为 timeline 的稳定排序键。这个设计几乎是对 SDK `stream_events()` 生命周期的前端映射。²⁴

工具与模型策略

function_tool 规范

SDK 的工具层推荐从 `@function_tool` 开始，并支持 docstring 解析、Pydantic `Field` 约束、异步超时、错误处理与 Agent-as-tool。对你的项目来说，最关键的工具规范有四条。第一，**参数必须可验证**，尽量用 Pydantic / `Field` 描述范围、枚举、pattern。第二，**工具必须区分读与写**。第三，**所有写工具默认不进 MVP，或必须审批**。第四，**工具必须带可审计元数据**，至少保留 `tool_call_id`、参数、输出、耗时、`run_id`。²⁵

规范项	建议	依据
参数校验	使用 <code>Field</code> / <code>Annotated</code> 限制范围与描述	<code>function_tool</code> 生成的 JSON schema 与验证会继承这些约束。 ²⁶
超时	异步工具统一配置 timeout	SDK 支持 <code>@function_tool(timeout=...)</code> ，超时可回模型或直接抛异常。 ²⁶
错误处理	自定义 <code>failure_error_function</code>	工具报错时返回可读错误，而不是整个 run 崩溃。 ²⁶
审计	保留 <code>tool_name/tool_call_id/tool_arguments</code>	<code>ToolContext</code> 原生暴露这些元数据。 ²⁷
候选工具集大小	主模型候选工具尽量少	Qwen 官方建议传给主模型的候选工具集控制在 20 个以内。 ¹²
危险工具	默认隔离，不直接暴露	Qwen 官方建议永远不要给模型直接暴露任意代码执行、危险 fs/db/payment 工具。 ¹²

建议的 MVP 工具清单可以很聚焦：

工具名	类型	读/写	来源
<code>check_sensitive_words</code>	电商规则校验	读	本地 Python
<code>extract_selling_points</code>	文本解析	读	本地 Python / 小模型
<code>search_tech_notes</code>	技术笔记检索	读	SQLite / 内置文档
<code>explain_error_signature</code>	错误模式解析	读	本地 Python
<code>generate_image</code>	图片生成	写外部服务但非破坏性	Provider adapter
<code>list_assets</code>	图片资产列表	读	SQLite
<code>save_contact</code>	高危写工具，v2 才加	写	需审批

安全与权限

安全部分建议直接用“防御纵深”原则来设计，而不是只写一句“注意 prompt injection”。OpenAI 的安全文档明确指出：prompt injection 是常见且危险的攻击方式，恶意内容可能来自用户输入、网页内容、文件检索结果甚至 MCP 搜索结果；缓解方法包括：不要把不可信输入放进 developer messages、在节点之间使用 structured outputs、用 guardrails 做输入筛查、让高风险工具保留人工审批、减少输入长度、限制输出 token，并始终做服务端校验与审计。 ²⁸

本项目建议落地为以下规则：

风险点	规则
Prompt injection	不可信内容只放 user message，不放 developer / system 模板
工具越权	读工具与写工具分离；Triage 默认只拿读工具
非结构化跨节点传播	Triage 与 Judge 统一使用结构化输出

风险点	规则
高危操作	写工具必须走审批；未来用 SDK HITL pause/resume
输入污染	加输入 guardrail：越权请求、越狱词、超长文本、恶意链接
输出失控	对 <code>JudgeReport</code> / <code>RouteDecision</code> / <code>ImagePlan</code> 做 schema 校验
日志泄露	raw prompt 默认只在 debug 模式可见；落库时脱敏
数据库风险	不向模型暴露原始 SQL；只暴露受控函数和白名单视图

SDK 自己也提供 guardrails 体系：input guardrails 可以在 agent 之前或并行运行，output guardrails 可检查最终输出，tool guardrails 可以在工具调用前后拦截并选择 allow / reject / raise exception。对于 MVP，你不需要一次全用上，但至少为“输入检查 + 工具审批”留出架构点位。 ²⁹

多模型接入策略

总体策略

推荐把所有第三方文本模型在 MVP 阶段统一接到 `OpenAIChatCompletionsModel`，而不是尝试混用 Responses 与 Chat Completions。理由有三点。第一，SDK 官方明确写到：许多第三方 provider 仍不完整支持 Responses，因此官方集成示例经常用 Chat Completions 形态。第二，GLM、MiniMax、Qwen 都有 OpenAI-compatible Chat Completions 端点。第三，Chat Completions 路径允许你借助 `ModelSettings.extra_body` 把 provider 的私有参数透传下去，例如 GLM 的 thinking、Qwen 的 `enable_thinking`、MiniMax 的 `reasoning_split` 等。 ³⁰

Provider 适配矩阵

Provider	接入形态	推荐配置	备注
GLM	<code>OpenAIChatCompletionsModel</code>	<code>base_url="https://open.bigmodel.cn/api/paas/v4/"</code> ，模型如 <code>glm-5.2</code> 、 <code>glm-5.1</code>	智谱提供 OpenAI 兼容接口； <code>glm-5.2</code> chat completions 支持流式与工具调用；文档示例也展示了通过 <code>extra_body</code> 传 thinking。 ³¹
MiniMax	<code>OpenAIChatCompletionsModel</code>	<code>base_url="https://api.minimax.io/v1"</code> ，模型如 <code>MiniMax-M3</code>	MiniMax 支持 OpenAI-compatible Chat Completions，也有 <code>/v1/responses</code> ；M3 支持 thinking、流式、图像/视频理解与 tool content。 ³²
Qwen	<code>OpenAIChatCompletionsModel</code>	北京区如 <code>base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"</code> ，模型如 <code>qwen-plus</code>	DashScope 支持 OpenAI-compatible；function calling 文档覆盖多步调用流程与工具最佳实践。 ³³

Provider	接入形态	推荐配置	备注
NVIDIA NIM LLM	OpenAIChatCompletionsModel	指向你的 NIM gateway .../ v1	NIM LLM 支持 /v1/ chat/completions、 streaming、tool calling、/v1/ models。 ³⁴
NVIDIA NIM Image	自定义 image adapter	OpenAI-compatible image generation endpoint	文档明确支持 flux.1- dev、 flux.1-schnell、 flux.2-klein-4b。 ³⁵
BFL FLUX	自定义 async image adapter	官方 FLUX API create + poll	官方文档指出其图像 API 采用异步设计，先 请求生成，再查询结 果。 ³⁶

模型配置格式

建议把模型池配置做成数据库驱动，而不是硬编码在 Python 里：

```
{
  "id": "glm_52",
  "display_name": "GLM-5.2",
  "provider": "zhipu",
  "modality": "text",
  "api_shape": "chat_completions",
  "base_url": "https://open.bigmodel.cn/api/paas/v4/",
  "model_id": "glm-5.2",
  "api_key_env": "ZAI_API_KEY",
  "enabled": true,
  "priority": 100,
  "capabilities": {
    "tools": true,
    "stream": true,
    "structured_output": true
  },
  "extra_body_defaults": {
    "thinking": {
      "type": "enabled"
    }
  },
  "timeout_ms": 30000
}
```

```
{
  "id": "qwen_plus",
  "display_name": "Qwen-Plus",
```



```

"provider": "dashscope",
"modality": "text",
"api_shape": "chat_completions",
"base_url": "https://dashscope.aliyuncs.com/compatible-mode/v1",
"model_id": "qwen-plus",
"api_key_env": "DASHSCOPE_API_KEY",
"enabled": true,
"priority": 90,
"extra_body_defaults": {
  "enable_thinking": true
}
}

```

`ModelSettings.extra_body` 在 SDK 里是正式字段；而 `OpenAIChatCompletionsModel` 参考实现也会把 `extra_body` 透传到底层请求体中。因此，把 provider 私有参数沉淀到 `model_configs.extra_body_defaults_json` 是合理且可维护的设计。 37

代码接入示例

单 provider 全局默认模式：

```

import os
from openai import AsyncOpenAI
from agents import set_default_openai_api, set_default_openai_client, set_tracing_disabled

# 没有 platform.openai.com key 时，先关闭默认 tracing
set_tracing_disabled(True)

client = AsyncOpenAI(
    api_key=os.getenv("ZAI_API_KEY"),
    base_url="https://open.bigmodel.cn/api/paas/v4/",
)

set_default_openai_client(client, use_for_tracing=False)
set_default_openai_api("chat_completions")

```

多 provider 按 agent 指定模式：

```

import os
from openai import AsyncOpenAI
from agents import Agent, ModelSettings, OpenAIChatCompletionsModel, set_tracing_disabled

set_tracing_disabled(True)

glm_client = AsyncOpenAI(
    api_key=os.getenv("ZAI_API_KEY"),
    base_url="https://open.bigmodel.cn/api/paas/v4/",
)

```

```

qwen_client = AsyncOpenAI(
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
)

triage_agent = Agent(
    name="TriageAgent",
    instructions="Route the task and respond concisely.",
    model=OpenAIChatCompletionsModel(model="glm-5.2", openai_client=glm_client),
    model_settings=ModelSettings(
        extra_body={"thinking": {"type": "enabled"}}
    ),
)

judge_agent = Agent(
    name="ModelJudgeAgent",
    instructions="Compare candidate answers and return a structured report.",
    model=OpenAIChatCompletionsModel(model="qwen-plus", openai_client=qwen_client),
    model_settings=ModelSettings(
        extra_body={"enable_thinking": True}
    ),
)

```

上面的模式分别对应 SDK 配置文档里的 `set_default_openai_client()` / `set_default_openai_api("chat_completions")`，以及模型文档里的“按 agent 直接传 concrete model object”。GLM、Qwen 的 `base_url` 与示例参数也都来自各自官方 OpenAI-compatible 文档。 ³⁸

路由、并发对比、超时与降级

多模型策略建议如下：

场景	路由策略
普通聊天	用户选中的 <code>primary_model_id</code> 直接执行
Auto 模式	Triage 判定任务类型后，按 provider capability 选模型
Compare 模式	对 <code>compare_model_ids</code> 并发 fan-out，再交给 Judge
图片生成	直接调用 <code>generate_image</code> tool，不走文本模型 route
某模型超时	标记该子 run <code>timed_out</code> ，Judge 仍基于剩余结果输出
某 provider 不支持 tool call	降级为“工具禁用模式”或切换兼容模型
usage 丢失	对流式 Chat Completions 明确开启 <code>include_usage=True</code>

SDK 的 usage 文档特别提醒：使用第三方 adapter / backend 时，usage 是否完整取决于 provider 是否返回；而对于 streamed Chat Completions，有时需要 `ModelSettings(include_usage=True)` 才能拿到 usage chunk。这个提醒对你的 compare 面板很关键，否则你可能只有答案、没有 token 成本。 ³⁹

验收、可观测性与演示

流式与可观测性

这个项目的核心呈现面，不是“模型答出了什么”，而是“系统是怎么跑出来的”。OpenAI Agents SDK 在这方面给了很完整的抓手：tracing 默认开启，会记录 LLM generations、tool calls、handoffs、guardrails 与 custom events；同时 SDK 还提供了两个 logger：openai.agents 与 openai.agents.tracing，并支持 enable_verbose_stdout_logging()。对于使用第三方 OpenAI-compatible provider 且没有 OpenAI 官方 key 的情况，官方明确建议关闭 tracing 或单独配置 tracing exporter key。40

本项目建议采用“两层可观测性”：

层级	记录内容
应用层	trace_id、thread_id、run status、SSE 事件、工具调用、用户 message、最终输出
SDK / provider 层	raw_responses、new_items、token usage、当前 agent、guardrail results

trace_id 建议由后端每次 run 启动时生成，thread_id 直接使用 conversation_id。虽然 SDK 文档没有强制统一的“thread_id”概念，但 session 本身就是会话上下文的天然锚点，因此用 conversation_id / session_id 作为 thread 标识最直观。与此同时，把 raw_responses 单独保存，可以在 provider 行为不一致、tool call payload 结构异常、usage 缺失时快速排查。41

MVP 验收标准

验收项	标准
基础会话	能创建、切换、回看会话，消息可持久化
路由能力	Auto 模式下，至少能在 Tech / Ecommerce / Image 三类中正确分流
流式输出	前端能实时显示 token；时间线能显示工具调用与 agent 更新
运行审计	每次 run 能落库 agent_runs、run_events、tool_calls
多模型接入	至少 2 个第三方文本模型可切换，且不依赖 OpenAI 官方 key
模型对比	至少支持 2 个模型并发比较，并输出可读 Judge 结论
图片生成	至少支持 1 个 FLUX 通道，生成后在画廊可见
可观测性	右侧调试面板能查看 raw response、usage、最终 agent、tool 参数
安全基础	高危写工具不进入 MVP；输入做最基本长度与模式校验
tracing 配置	无 OpenAI key 情况下能正常运行，不因 tracing 报错

这些验收项之所以合理，是因为它们分别对应了 SDK 的核心结果面、streaming 事件、usage 统计和 tracing/配置能力。42

面试演示脚本

场景	演示输入	预期 Agent 行为	前端展示	数据库证据
电商文案检测并优化	“把这条标题改得更像天猫风格，并检查违规词：史上最低！全网第一！绝对治愈失眠…”	Triage → EcommerceAgent → <code>check_sensitive_words</code> → 重写文案 → 输出 before/after	聊天区显示优化结果；时间线显示 tool.called/ tool.output	<code>tool_calls</code> 有 <code>check_sensitive_words</code> ; <code>agent_runs.final_agent=EcommerceAgent</code>
技术报错解释	“解释这个错误：TypeError: object NoneType can't be used in 'await' expression”	Triage → TechAgent → <code>explain_error_signature</code> / <code>search_tech_notes</code> → 结构化修复建议	主区显示原因、定位与修复；右侧显示工具参数和输出	<code>run_events</code> 有 <code>tool.called</code> 记录； <code>messages</code> 保存最终解释
同题多模型对比	“比较 GLM、MiniMax、Qwen 写一段 618 活动文案，给我最终推荐”	Triage → compare pipeline → 并发子 runs → ModelJudgeAgent 输出评分卡	ComparePanel 显示三个答案、耗时、token、结论	<code>agent_runs</code> 有 parent/child runs ; <code>usage_json</code> 完整
图片生成并回顾过程	“生成一张极简深色科技风的 AI 工作台海报”	Triage → ImageAgent → <code>generate_image</code> → 保存资产	聊天区出现图片卡片；画廊新增缩略图；时间线显示 image tool	<code>generated_assets</code> 新增一条； <code>tool_calls.tool_name=generate_image</code>

这四个场景分别覆盖了：规则工具 + 文案优化、技术专家工具、并发比较 + LLM judge、外部图片工具 + 资产落库，足以构成一套有层次的面试演示路径。与此同时，它们都能直接映射到 SDK 官方 examples 中的 routing、agents-as-tools、parallel execution、LLM as judge、image tool output 这些模式。 7

开发计划、风险与参考

里程碑

建议按四阶段、八周推进。这样既适合边做边学，也更容易把每个里程碑包装成可演示成果。

阶段	时间	主要产出	验收标准
阶段一	第 1–2 周	React 聊天骨架、会话列表、FastAPI 基础接口、SQLite 表、单模型文本聊天	能完成创建会话、发送问题、流式显示回答、消息落库
阶段二	第 3–4 周	TriageAgent、Tech/Ecommerce/Image 三类 specialist、工具链、SSE 事件时间线	能看到 agent.updated、tool.called、tool.output、run.completed
阶段三	第 5–6 周	多模型比较、ModelJudgeAgent、usage 与 raw_responses 面板、图片生成与画廊	compare 模式可并发跑至少两个模型；图片可生成并持久化
阶段四	第 7–8 周	guardrails、审批预留、调试抽屉完善、演示脚本、README/部署脚本	能完整演示 3–4 个场景，并给出技术讲解路线

风险与注意事项

风险	影响	缓解策略
第三方 provider 兼容性不完全一致	tool call / usage / stream payload 可能不统一	MVP 统一走 <code>OpenAIChatCompletionsModel</code> ；provider 私有参数通过 <code>extra_body</code> 管理；逐个验证能力矩阵。 ⁴³
无 OpenAI 官方 key 导致 tracing 报错	项目本地无法启动或噪音过多	启动时 <code>set_tracing_disabled(True)</code> ，或单独配置 tracing key。 ⁴⁴
流式 usage 缺失	Compare 视图缺少成本维度	对 streamed Chat Completions 打开 <code>include_usage=True</code> ，并验证每个 provider。 ³⁹
图片 provider 差异大	FLUX 生成接口不统一	MVP 优先选 NVIDIA NIM image endpoint；BFL 走独立 async adapter。 ¹³
Prompt injection	越权工具调用、泄露数据、错误行为	不可信输入不进 developer message；structured outputs；guardrails；高危工具审批；服务端验证。 ²⁸
SQLite 并发写限制	多用户下锁冲突	MVP 可接受；v1/v2 迁移 PostgreSQL 或 SQLAlchemySession。 ⁴⁵
Tool surface 过大	延迟升高，工具选择错误率升高	候选工具集做分层筛选，主模型候选工具控制在小集合。 ¹²

推荐技术栈

层	推荐
前端	React + TypeScript + Vite + TanStack Query + Zustand + 组件库
后端	FastAPI + Pydantic v2 + SQLAlchemy 2.0 + <code>openai-agents</code>
存储	SQLite（MVP），保留迁移 PostgreSQL 的接口

层	推荐
流式	SSE（MVP），WebSocket（v2 审批/双向交互）
模型池	GLM、MiniMax、Qwen 文本模型；NVIDIA NIM / BFL FLUX 图片
日志/调试	Python logging + <code>openai.agents</code> / <code>openai.agents.tracing</code> logger
部署	Docker Compose；后续拆分前后端与对象存储

参考资料

以下资料最值得直接纳入 README 的“设计依据 / 学习路径”部分：

- OpenAI Agents SDK：Quickstart、Agents、Tools、Running agents、Streaming、Results、Models、Configuration、Tracing、Guardrails、Sessions、Context、Examples。 ⁴⁶
- 智谱 GLM OpenAI-compatible 与 function calling 文档。 ⁴⁷
- MiniMax OpenAI-compatible Chat Completions / Responses / Tool Use 文档。 ⁴⁸
- 阿里云 Qwen OpenAI-compatible 与 Function Calling 文档。 ⁴⁹
- NVIDIA NIM OpenAI-compatible LLM 与 Image Generation 文档。 ⁵⁰
- Black Forest Labs FLUX 官方 API 文档。 ³⁶
- SSE / EventSource 参考资料。 ¹⁸
- OpenAI 安全文档：Safety in building agents、Safety best practices、Security & Privacy。 ²⁸

这份需求文档对应的最佳实现路线，不是一性把所有高级能力都塞满，而是先把 **“Triage + Tools + Stream + RunResult 落库 + Debug 面板 + 多模型接入”** 这条主线做扎实。因为这条主线已经足够全面地体现你对 OpenAI Agents SDK、OpenAI-compatible 模型接入、Agent 可观测性、工具治理和系统设计的理解。

¹ ² ⁵ ⁸ ⁹ ¹⁰ <https://openai.github.io/openai-agents-python/agents/>
<https://openai.github.io/openai-agents-python/agents/>

³ ³¹ ⁴⁷ <https://docs.bigmodel.cn/cn/guide/develop/openai/introduction>
<https://docs.bigmodel.cn/cn/guide/develop/openai/introduction>

⁴ ⁷ ¹⁵ <https://openai.github.io/openai-agents-python/examples/>
<https://openai.github.io/openai-agents-python/examples/>

⁶ ¹⁶ ⁴² <https://openai.github.io/openai-agents-python/results/>
<https://openai.github.io/openai-agents-python/results/>

¹¹ ¹⁹ https://openai.github.io/openai-agents-python/running_agents/
https://openai.github.io/openai-agents-python/running_agents/

¹² <https://help.aliyun.com/en/model-studio/qwen-function-calling>
<https://help.aliyun.com/en/model-studio/qwen-function-calling>

¹³ ³⁵ Image Generation API (OpenAI-Compatible) — NVIDIA NIM for Visual Generative AI (GenAI)
<https://docs.nvidia.com/nim/visual-genai/latest/api/openai-image-generation.html>

¹⁴ ²¹ ²⁴ Streaming - OpenAI Agents SDK
<https://openai.github.io/openai-agents-python/streaming/>

¹⁷ ⁴¹ <https://openai.github.io/openai-agents-python/sessions/>
<https://openai.github.io/openai-agents-python/sessions/>

- 18 20 <https://developer.mozilla.org/en-US/docs/Web/API/EventSource>
<https://developer.mozilla.org/en-US/docs/Web/API/EventSource>
- 22 <https://openai.github.io/openai-agents-python/ref/memory/>
<https://openai.github.io/openai-agents-python/ref/memory/>
- 23 27 <https://openai.github.io/openai-agents-python/context/>
<https://openai.github.io/openai-agents-python/context/>
- 25 26 <https://openai.github.io/openai-agents-python/tools/>
<https://openai.github.io/openai-agents-python/tools/>
- 28 <https://developers.openai.com/api/docs/guides/agent-builder-safety>
<https://developers.openai.com/api/docs/guides/agent-builder-safety>
- 29 <https://openai.github.io/openai-agents-python/guardrails/>
<https://openai.github.io/openai-agents-python/guardrails/>
- 30 43 44 **Models - OpenAI Agents SDK**
<https://openai.github.io/openai-agents-python/models/>
- 32 <https://platform.minimax.io/docs/token-plan/other-tools>
<https://platform.minimax.io/docs/token-plan/other-tools>
- 33 49 <https://help.aliyun.com/zh/model-studio/compatibility-of-openai-with-dashscope>
<https://help.aliyun.com/zh/model-studio/compatibility-of-openai-with-dashscope>
- 34 50 **API Reference — NVIDIA NIM for Large Language Models 2.0**
<https://docs.nvidia.com/nim/large-language-models/2.0.0/reference/api-reference.html>
- 36 **Image Generation with Text Prompts - Black Forest Labs**
https://docs.bfl.ai/quick_start/generating_images
- 37 https://openai.github.io/openai-agents-python/ref/model_settings/
https://openai.github.io/openai-agents-python/ref/model_settings/
- 38 **Configuration - OpenAI Agents SDK**
<https://openai.github.io/openai-agents-python/config/>
- 39 <https://openai.github.io/openai-agents-python/usage/>
<https://openai.github.io/openai-agents-python/usage/>
- 40 **Tracing - OpenAI Agents SDK**
<https://openai.github.io/openai-agents-python/tracing/>
- 45 **SQLAlchemy session - OpenAI Agents SDK**
https://openai.github.io/openai-agents-python/sessions/sqlalchemy_session/
- 46 **Quickstart - OpenAI Agents SDK**
https://openai.github.io/openai-agents-python/quickstart/?utm_source=chatgpt.com
- 48 **Chat Completions API - MiniMax API Docs**
<https://platform.minimax.io/docs/api-reference/text-chat-openai>